Predictive Modeling of Fouling Resistance in Shell-and-Tube Heat Exchangers

A Data-Driven Approach to Predictive Maintenance

Author: Abhiraj Mandal

Roll No.: 230107005

Status: Pre-final yearite

Institute: Indian Institute of Technology Guwahati

Repository: https://github.com/DataWorshipper/Fouling-Prediction

Live App: https://fouling-prediction-f9voxwrzp9teuyn65q93wp.streamlit.app/

April 23, 2026

0 Contents

1 Abstract **4**

2 Problem Context and Physical Principles **4**

2.1 The Physics of Heat Exchanger Fouling 4

2.2 The Sawtooth Degradation Cycle 5

2.3 Heat Duty and Energy Balance 5

2.4 The Pressure Drop–Fouling Interaction 6

3 Dataset Description **6**

3.1 Source and Scale 6

3.2 Column Definitions and Physical Significance 6

3.3 Engineered Features 9

4 Computational Framework **9**

4.1 Why Polars Over Pandas? 9

4.2 Modeling Strategy: Bagging vs. Boosting 10

5 Data Acquisition, Preprocessing, and Cleaning **10**

5.1 Step 1: Data Loading and Schema Validation 10

5.2 Step 2: Schema Transformation and Standardisation 10

5.3 Step 3: Thermodynamic Standardisation — The Kelvin Standard 11

5.4 Step 4: Initial Feature Pruning — Removing Design Constants 11

5.5 Step 5: Missing Value Audit 11

5.6 Step 6: Skewness Analysis 12

5.7 Step 7: Selective Z-score Standardisation 12

6 Exploratory Data Analysis (EDA) **13**

6.1 6.1 Visualising the Sawtooth Degradation Cycle 13

6.2 6.2 CIP Interval Distribution — Reactive vs. Periodic Maintenance 14

6.3 6.3 Physical Drift: Flow Rate vs. Pressure Drop 16

6.4 6.4 Fouling Kinetics: dR_f/dt by Scenario 17

6.5 6.5 Target Correlation Profiling 18

6.6 6.6 Variance Inflation Factor (VIF) Analysis 19

7 Time-Series Analysis and Feature Engineering **19**

7.1 7.1 Augmented Dickey-Fuller Test — Confirming Non-Stationarity 19

7.2 7.2 STL Decomposition — Separating Trend from Maintenance Cycles . . 20

7.3 7.3 ACF and PACF Analysis — Determining Lag Structure 21

7.4 7.4 Engineered Temporal Features 22

8 Baseline Statistical Modeling: ARIMA and SARIMA **22**
8.1 8.1 ARIMA(1, 1, 0) — Global Baseline . . . 22
8.2 8.2 SARIMA(1,1,0)(0,1,0)$_{25}$ — Cyclic Baseline . . . 23
8.3 8.3 Baseline Results and the Complexity Gap . . . 23

9 Non-Linear Ensemble Modeling with Optuna HPO **24**
9.1 9.1 Train-Test Split Strategy . . . 24
9.2 9.2 Optuna Hyperparameter Optimisation Framework . . . 24
9.3 9.3 Individual Model Tuning . . . 24
9.3.1 XGBoost . . . 24
9.3.2 CatBoost . . . 24
9.3.3 Random Forest and AdaBoost . . . 25
9.4 9.4 Final Performance Leaderboard . . . 25
9.5 9.5 Analysis of Results . . . 25

10 Deep Learning Architectures **26**
10.1 10.1 Motivation: Why Deep Learning? . . . 26
10.2 10.2 Long Short-Term Memory (LSTM) . . . 26
10.3 10.3 Temporal Fusion Transformer (TFT) . . . 27
10.4 10.4 Final Comprehensive Leaderboard . . . 27

11 GitHub Repository Structure **28**
11.1 `/notebooks` — Research and Experimentation . . . 28
11.2 `/data` — Dataset Management . . . 28
11.3 `/etl` — Extract, Transform, Load Pipeline . . . 29
11.4 `/model` — Model Training Scripts . . . 29
11.5 `/artifacts` — Serialised Model Objects . . . 29
11.6 `/config` — Configuration Management . . . 29
11.7 `/app` — Streamlit Deployment . . . 30

12 Streamlit Application and Deployment **30**
12.1 Application Overview . . . 30
12.2 Architecture . . . 30

13 Scalability Analysis and Areas for Improvement **30**
13.1 Current System Limitations . . . 31
13.1.1 Static Scaler — Training-Serving Skew Risk . . . 31
13.1.2 Scenario-Level Training — Single Equipment Unit . . . 31
13.1.3 Point Predictions Without Uncertainty . . . 31
13.1.4 No Streaming / Real-Time Inference Pipeline . . . 31
13.2 Recommended Improvements . . . 31

13.2.1 Online Learning and Continual Model Updating 31
13.2.2 Physics-Informed Neural Networks (PINNs) 32
13.2.3 Anomaly Detection Layer 32
13.2.4 Containerisation and MLOps 32
13.2.5 Database Integration 32

14 Conclusion **32**

A Model Performance Summary Table **33**

B Final Feature Set **34**

1 Abstract

Fouling is a critical operational bottleneck in heat transfer processes, characterised by the accumulation of unwanted deposits on heat exchanger surfaces. This phenomenon introduces an additional thermal resistance, R_f, which progressively reduces the Overall Heat Transfer Coefficient, U, causing energy inefficiency and unplanned downtime. In large-scale industrial plants, the economic cost of unaddressed fouling — through increased pumping power, reduced throughput, and emergency shutdowns — can run into millions of dollars annually.

This project moves away from traditional reactive and periodic maintenance schedules and toward a fully data-driven **Predictive Maintenance** framework. Using a large-scale industrial simulation dataset comprising over 700,000 observations spread across 80 distinct operational scenarios, a series of progressively sophisticated models are developed and evaluated.

The study begins with classical statistical time-series models (ARIMA, SARIMA), establishes the need for exogenous, physics-informed features, and then transitions to non-linear ensemble methods optimised via the Optuna hyperparameter tuning framework. Finally, deep learning architectures — specifically Long Short-Term Memory (LSTM) networks and the Temporal Fusion Transformer (TFT) — are implemented to explore whether learned temporal representations can surpass engineered features.

The key finding is that **XGBoost**, augmented with domain-engineered temporal features, achieves near-perfect prediction accuracy ($R^2 = 0.9996$, MAE $= 0.0041$), significantly outperforming all baselines. The full pipeline is packaged into a production-grade repository and deployed as an interactive web application.

2 Problem Context and Physical Principles

2.1 The Physics of Heat Exchanger Fouling

A shell-and-tube heat exchanger transfers thermal energy between two fluid streams — one flowing through the tubes (tube side) and one flowing around the outside of the tubes in the shell (shell side). The efficiency of this transfer is governed by the **Overall Heat Transfer Coefficient**, U (units: $\mathrm{W/m^2 \cdot K}$). In a pristine, clean exchanger, U is at its design-stage maximum, denoted U_{clean}.

As the exchanger operates over time, mineral deposits, biological growth, corrosion products, or particulate matter accumulate on the inner surfaces of the tubes. This layer acts as an insulating blanket, introducing an additional thermal resistance — the

Fouling Resistance, R_f (units: $\mathrm{m}^2 \cdot \mathrm{K/W}$). The relationship between the clean and fouled performance is:

$$\frac{1}{U_{\text{dirty}}} = \frac{1}{U_{\text{clean}}} + R_f \tag{1}$$

This equation is the cornerstone of this project. As R_f increases, U_{dirty} decreases, meaning more energy is required to achieve the same heat duty. The primary objective is to predict $R_f(t)$ — the fouling resistance as a function of time — so that maintenance can be scheduled before U_{dirty} falls below a critical operational threshold.

2.2 The Sawtooth Degradation Cycle

Fouling is not a monotonic, linear increase. Industrial heat exchangers undergo periodic **Cleaning-In-Place (CIP)** events, during which the exchanger is chemically or mechanically cleaned. This resets R_f to approximately zero. The result is a characteristic **"sawtooth" pattern** in the time-series of R_f:

- **Induction/Deposition Phase:** R_f increases as deposits adhere to tube surfaces. The rate (dR_f/dt) may be roughly linear early on (induction period) but eventually slows as the deposit layer thickens, increasing the removal force from the flowing fluid (asymptotic fouling).
- **CIP Event:** A discrete maintenance action causes a near-instantaneous drop in R_f toward zero.

This pattern has profound modeling implications. Standard regression models assume a continuous, stationary relationship between inputs and output. The sawtooth violates this assumption because: (1) the process is non-stationary — its statistical properties change after each reset; (2) the process exhibits structural breaks; and (3) the correct prediction of R_f depends on knowing *where in the cycle* the exchanger currently is, not just the instantaneous sensor readings.

2.3 Heat Duty and Energy Balance

The actual heat transferred by the exchanger, Q (Watts), is governed by the energy balance:

$$Q = U_{\text{dirty}} \cdot A \cdot \Delta T_{\text{lm}} \tag{2}$$

where A is the heat transfer area (m^2) and ΔT_{lm} is the Log Mean Temperature Difference (LMTD). As fouling increases R_f, U_{dirty} falls, and either Q falls (heat duty deficit) or

operators must increase the inlet temperature or flow rate to compensate — both of which increase operating costs.

2.4 The Pressure Drop–Fouling Interaction

A key physical relationship exploited in the feature engineering phase is the interaction between mass flow rate and pressure drop. In a clean tube, the Darcy-Weisbach equation gives:

$$\Delta P = f \cdot \frac{L}{D} \cdot \frac{\rho v^2}{2} \tag{3}$$

As fouling deposits accumulate, the effective inner diameter D decreases. For a constant mass flow rate, the fluid velocity v increases, and consequently ΔP increases. This means that *at the same flow rate, a fouled exchanger shows higher pressure drop than a clean one* — a physical signature that the data confirms.

3 Dataset Description

3.1 Source and Scale

The dataset is sourced from the publicly available Kaggle simulation dataset: *"Shell-and-Tube Heat Exchanger Fouling Simulation"* by F. David Santillan. It contains **over 700,000 hourly observations** across **80 distinct operational scenarios**, each representing a different combination of inlet conditions, flow rates, and fouling types (e.g., scaling, biological, particulate).

3.2 Column Definitions and Physical Significance

The raw column names from the simulation dataset are abbreviated and inconsistent. Table 1 provides the standardised nomenclature adopted in this project, along with the physical meaning and units of each variable.

Table 1: Dataset Column Definitions After Schema Transformation

Column Name	**Units**	**Physical Significance**
`scenario`	—	Integer identifier for each of the 80 unique operating scenarios. Each scenario represents a distinct set of inlet conditions and fouling type. Used for data partitioning (train/test split) rather than as a model feature.
`time_hours`	hours	Cumulative operational time elapsed since the start of the current scenario. Acts as the primary temporal index.
`T_in_actual_K`	Kelvin	The measured inlet temperature of the tube-side fluid, converted from Celsius. Operating in Kelvin ensures thermodynamic consistency with heat transfer formulae (e.g., convective heat transfer coefficients depend on absolute temperature through fluid viscosity).
`flow_actual`	kg/s	The measured mass flow rate of the tube-side fluid. Higher flow rate increases the wall shear stress on the tube surface, which can limit deposit adhesion and reduce fouling rates.
`fluid_velocity`	m/s	Mean fluid velocity in the tube. Derived from `flow_actual`, tube cross-section, and fluid density. A higher velocity increases the Reynolds number and the wall shear stress. *Dropped after VIF analysis due to perfect collinearity with `flow_actual`.*
`reynolds_number`	—	Dimensionless number characterising the flow regime. $Re = \rho v D/\mu$. In turbulent regimes ($Re > 10{,}000$), mixing is high and fouling kinetics differ from laminar flow. *Dropped due to deterministic collinearity with velocity.*

(Table 1 continued)

Column Name	Units	Physical Significance
`shear_stress`	Pa	Wall shear stress (τ_w) — the tangential force per unit area exerted by the fluid on the tube wall. Governs the balance between deposition and removal of fouling deposits. *Dropped due to collinearity.*
`pressure_drop`	Pa	Pressure difference between the inlet and outlet of the tube side. A rising pressure drop at constant flow rate is a direct physical indicator of tube restriction caused by fouling deposit buildup. *Dropped after collinearity check but its relationship with fouling is validated in EDA.*
`U_actual_measured`	W/m^2K	The real-time measured Overall Heat Transfer Coefficient. Directly related to R_f via Eq. 1. *Dropped to prevent target leakage.*
`heat_duty_actual`	W	Measured heat transferred. Derived from U_{dirty}, hence also leaks the target. *Dropped.*
`efficiency_thermal`	fraction	The ratio of actual to design-clean heat duty. Provides an indication of degradation but is derived from U values. *Dropped.*
`R_wall`	m^2K/W	Thermal resistance of the tube wall itself (a fixed, static property of the exchanger geometry and material). Retained as a static covariate in the TFT model.
`is_cleaning`	binary (0/1)	A flag indicating whether a Cleaning-In-Place (CIP) event occurred at this time step. This is the most important contextual feature, as it marks the sawtooth resets.
`clean_efficiency`	fraction	Effectiveness of the CIP event (1.0 = full restoration to clean state). Retained as additional context for post-cleaning behavior.

(Table 1 continued)

Column Name	Units	Physical Significance
`target_Rf`	m^2K/W	**The prediction target.** The instantaneous fouling resistance. This is the value we seek to predict at every time step.

3.3 Engineered Features

Beyond the raw columns, three critical features are created through domain-informed feature engineering (Section 7):

Engineered Feature	Physical Rationale
`target_Rf_lag_1`	The fouling resistance at the previous time step. Captures the dominant AR(1) behavior identified in PACF analysis.
`target_Rf_rolling_12h`	12-hour rolling mean of R_f. Acts as a noise-resistant estimate of the current fouling "state".
`hours_since_clean`	Time elapsed since the last CIP event. This is the most important engineered feature — it tells the model where the exchanger is in the current fouling cycle, enabling mapping of sensor readings to a specific stage of the degradation curve.

4 Computational Framework

4.1 Why Polars Over Pandas?

The dataset exceeds 700,000 observations, making the choice of data manipulation library a genuine engineering concern. Pandas, the conventional choice, operates on a single-threaded, eager execution model. Every operation is executed immediately in sequence, with intermediate results materialised in memory. For this dataset size, this approach leads to significant memory overhead and slow execution.

Polars is chosen for its fundamentally different systems design:

- **Lazy Evaluation:** Polars builds a logical query plan rather than executing commands line-by-line. This enables *predicate pushdown* (filtering data before loading unnecessary rows) and *projection pushdown* (only loading the columns actually

needed).

- **SIMD Parallelism:** Polars uses the Apache Arrow columnar memory format and leverages SIMD (Single Instruction, Multiple Data) CPU instructions to process multiple data points simultaneously across all available CPU cores.
- **Rust Backend:** The core engine is implemented in Rust, providing memory safety and performance without garbage collection pauses.

In practice, this means operations on the full 700k-row dataset that might take 10–20 seconds in Pandas complete in under 2 seconds in Polars, with approximately 40–60% lower peak memory usage. For a Kaggle environment with limited RAM, this is critical for avoiding out-of-memory crashes.

4.2 Modeling Strategy: Bagging vs. Boosting

Rather than relying on a single algorithm, ensemble techniques are used to capture the complex, non-linear dynamics of heat transfer degradation.

Bagging (Bootstrap Aggregating) — implemented via Random Forest — reduces variance. Multiple decision trees are trained on different bootstrap subsamples of the data, and their predictions are averaged. This improves robustness against localised sensor noise and overfitting to specific operating conditions.

Boosting — implemented via XGBoost, CatBoost, and AdaBoost — operates iteratively. Each successive model focuses on correcting the residual errors of the previous one. This is particularly well-suited to capturing the sharp, non-linear transitions (sawtooth resets) and the asymptotic fouling kinetics that dominate this problem.

5 Data Acquisition, Preprocessing, and Cleaning

5.1 Step 1: Data Loading and Schema Validation

The dataset is downloaded via `kagglehub` and loaded with `pl.read_csv`. Although a production pipeline would use `pl.scan_csv` to maintain a lazy execution model, eager loading is used here to validate the full schema in memory before applying any transformations. This ensures that column names, data types, and overall integrity are verified before any downstream logic is applied — a critical practice when the raw dataset comes from an external source with abbreviated and inconsistent tags.

5.2 Step 2: Schema Transformation and Standardisation

Raw industrial simulation tags use abbreviated, inconsistent naming (e.g., `T_in_C`, `m_dot_nominal_kg_s`, `Rf_m2K_W`). A comprehensive rename map is applied to translate these into descriptive,

standardised names. The key principle is to create a clear distinction between:

- **Design-stage setpoints** (e.g., `U_design_clean`): the values for which the exchanger was designed.
- **Real-time sensor measurements** (e.g., `U_actual_measured`): what is actually happening during operation.

This distinction is not just cosmetic — it is essential for preventing target leakage, as design values are not available at prediction time in a real deployment.

5.3 Step 3: Thermodynamic Standardisation — The Kelvin Standard

All temperature features are converted from Celsius to Kelvin:

$$T_K = T_C + 273.15 \tag{4}$$

This is physically mandated. Heat transfer correlations (e.g., the Dittus-Boelter equation for convective heat transfer) and fluid properties such as viscosity and thermal conductivity are functions of absolute temperature. Using Celsius would introduce a non-physical offset in temperature-dependent calculations and could cause the model to learn incorrect relationships between temperature and fouling rate.

5.4 Step 4: Initial Feature Pruning — Removing Design Constants

Static design parameters (e.g., `T_in_design_C`, `T_in_design_K`, `flow_design`) are removed. These are fixed setpoints that do not vary across time steps within a scenario. Including them in the feature set provides no temporal signal and inflates the feature count, increasing training time without improving predictive power. Their presence also creates perfect multicollinearity with any scenario-level fixed effect.

5.5 Step 5: Missing Value Audit

A comprehensive null-count audit is performed across all columns. The dataset is verified to have **zero missing values**, which is expected given its simulation origin. This step is documented explicitly because in a real industrial deployment, sensor dropouts and communication failures are common, and the absence of a robust imputation strategy can cause silent model failures.

5.6 Step 6: Skewness Analysis

The skewness coefficient of each numeric feature is computed:

$$\text{Skewness} = \frac{\mathbb{E}\left[(X-\mu)^3\right]}{\sigma^3} \tag{5}$$

A value of 0 indicates a symmetric distribution; values above $|1|$ indicate significant asymmetry. The meta-analysis of the skewness distribution (Figure 1) reveals that the feature space is heterogeneous — some physical variables such as R_f and pressure drop follow heavy-tailed distributions, while others such as temperature and flow rate are approximately symmetric.

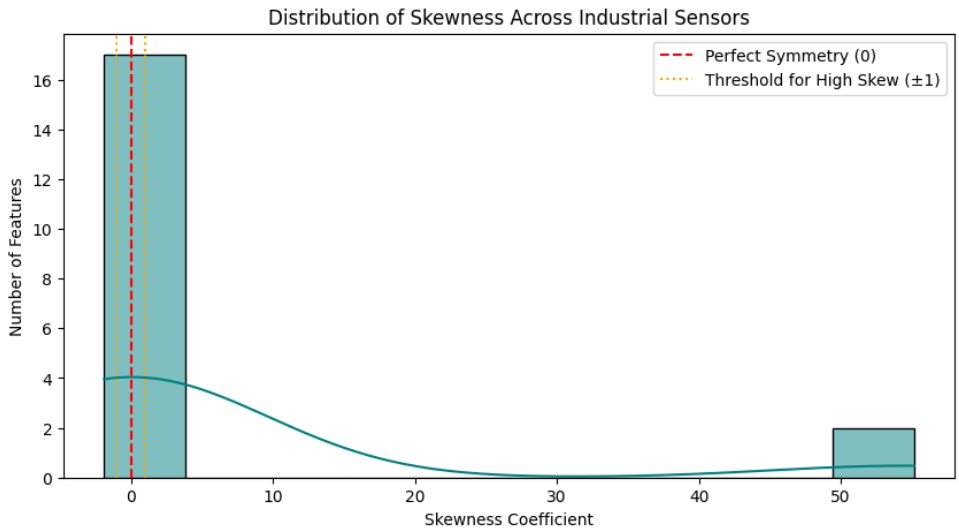

Figure 1: Distribution of skewness coefficients across all numeric features. The heterogeneity in skewness shapes justifies the use of Z-score standardisation over Min-Max scaling, as the latter would compress variation in the majority of symmetric features to accommodate the heavy-tailed minority.

5.7 Step 7: Selective Z-score Standardisation

Z-score standardisation is applied selectively, only to physical process sensors:

$$z = \frac{x-\mu}{\sigma} \tag{6}$$

The following features are *explicitly excluded* from scaling:

- `scenario`: Scaling a categorical identifier is meaningless and breaks grouping operations.
- `time_hours`: The primary temporal variable. Standardising it removes its physical meaning and makes rate-of-change calculations uninterpretable.

- `is_cleaning`: Already binary (0/1). Scaling would corrupt its role as a logical flag.
- `clean_efficiency`: A bounded fraction; scaling is unnecessary and could push values outside interpretable bounds.

Z-score vs. Min-Max: Z-score standardisation is chosen over Min-Max scaling specifically because of the CIP events. These events create extreme transient values in several sensor channels (sharp spikes in pressure drop at the moment of cleaning, for example). In Min-Max scaling, these extremes define the $[0, 1]$ range, compressing all normal operating data into a narrow interval and destroying meaningful variation. Z-score scaling, being robust to such transients, preserves the signal in steady-state operating conditions.

Industrial parameters span multiple orders of magnitude ($Re \sim 10^5$, $R_f \sim 10^{-4}$). Without standardisation, gradient-based optimisers and distance-based algorithms would heavily weight high-magnitude features, effectively ignoring low-magnitude but critical signals like fouling resistance.

6 Exploratory Data Analysis (EDA)

The EDA phase goes beyond standard summary statistics. It is designed to *physically validate* the dataset — to confirm that the patterns observed in the data are consistent with the engineering principles governing heat exchanger fouling. This validation is essential before trusting any model trained on the data.

6.1 6.1 Visualising the Sawtooth Degradation Cycle

Figure 2 shows the temporal profile of standardised fouling resistance for a single scenario. The CIP events (green dashed vertical lines) produce the characteristic sawtooth pattern: a gradual rise in R_f followed by a near-instantaneous reset.

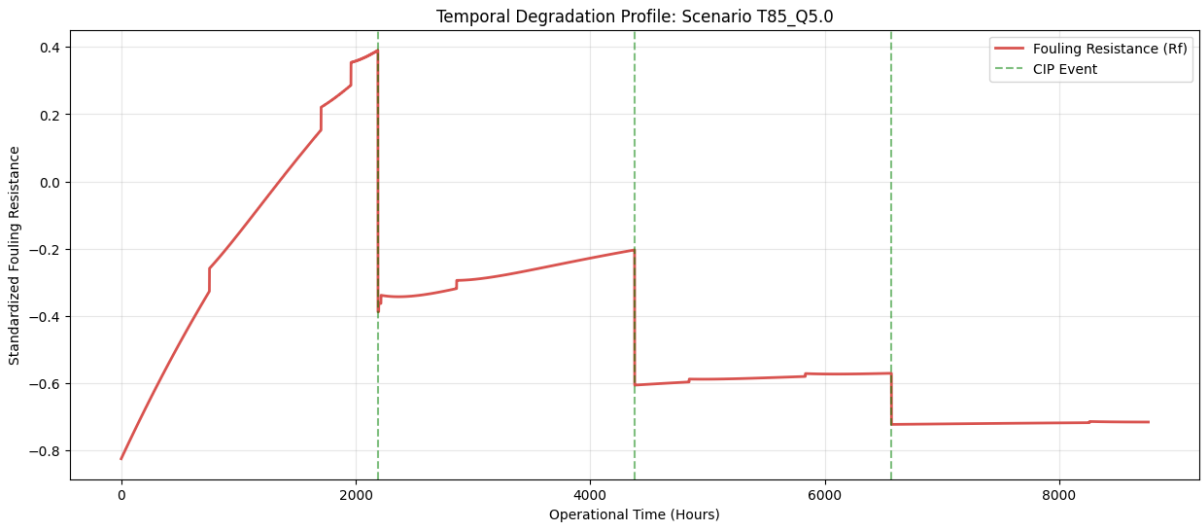

Figure 2: Temporal degradation profile for a single scenario. The sawtooth pattern is clearly visible — fouling resistance (R_f) increases steadily between Cleaning-In-Place events (green dashed lines) and resets sharply upon cleaning.

Why This Disqualifies Linear and Classical Statistical Models

This figure is the single most important justification for our modeling choices. Linear regression assumes a consistent, monotonic trend. ARIMA models assume stationarity (or achieve it through differencing). Neither assumption holds here:

- **Non-monotonicity:** R_f periodically returns to near-zero, violating any globally monotonic trend assumption.
- **Non-stationarity:** The mean and variance of R_f change abruptly after each reset.
- **Structural breaks:** Each CIP event is a structural break. A global linear model would average over these breaks, systematically underestimating R_f in the fouled state.

6.2 6.2 CIP Interval Distribution — Reactive vs. Periodic Maintenance

Figure 3 shows the distribution of time intervals between consecutive CIP events across all scenarios.

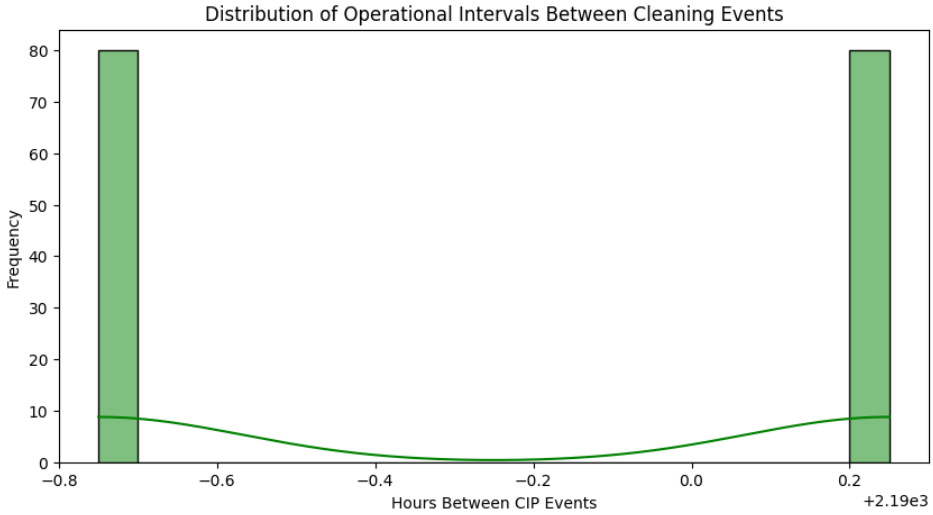

Figure 3: Distribution of operational intervals between Cleaning-In-Place events. The spread of the distribution reveals whether maintenance is periodic (fixed schedule) or reactive (triggered by performance degradation).

A wide, multi-modal distribution indicates that CIP events are not perfectly periodic — they are condition-driven. This has direct modeling implications: a SARIMA model that assumes a fixed seasonal period will be misaligned with actual cleaning events whenever those events deviate from the assumed period. The model will predict a reset at the "wrong" time, generating large errors. This is precisely why the SARIMA model — despite better MAE — achieves a dramatically negative R^2 (see Section 8).

6.3 6.3 Physical Drift: Flow Rate vs. Pressure Drop

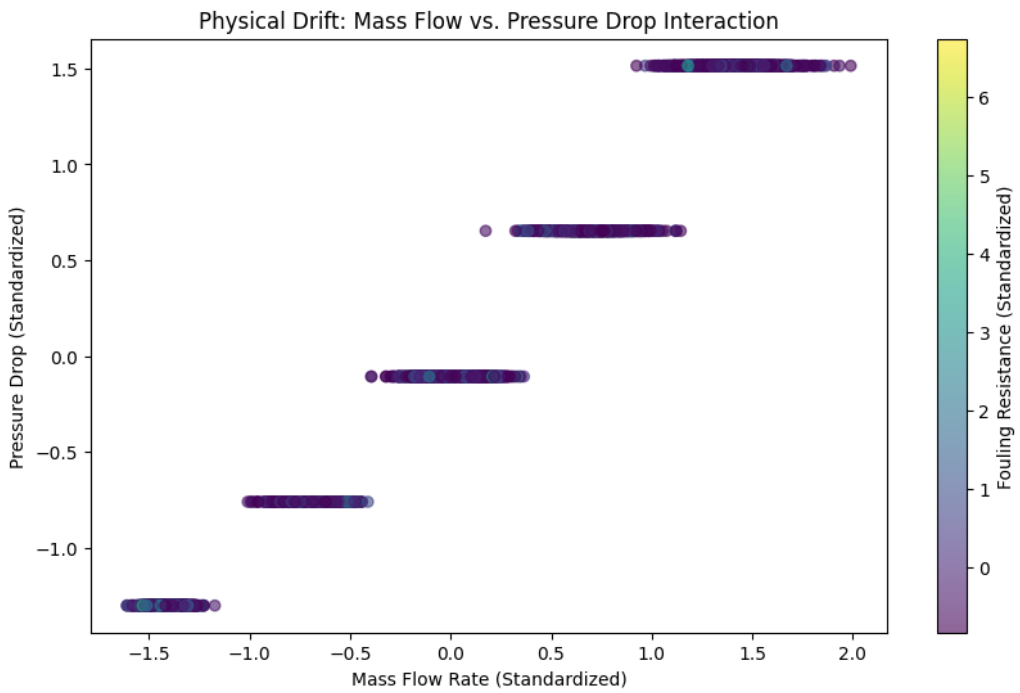

Figure 4: Scatter plot of mass flow rate vs. pressure drop, colour-coded by fouling resistance. Physical drift is evident: for the same mass flow rate, fouled exchangers (high R_f, yellow) exhibit higher pressure drop than clean ones (low R_f, purple), consistent with reduced effective tube diameter due to deposit buildup.

Figure 4 provides direct physical validation. In a perfectly clean exchanger, pressure drop and flow rate should follow a deterministic curve. The visible "cloud" of points at each flow rate value confirms physical drift — the pressure drop for a given flow rate is not constant but varies with the internal fouling state. The colour gradient shows that this variability is *caused by* fouling resistance. This validates the inclusion of both `flow_actual` and `pressure_drop` as informative model features, as their joint behavior encodes the internal state of the exchanger.

6.4 6.4 Fouling Kinetics: dR_f/dt by Scenario

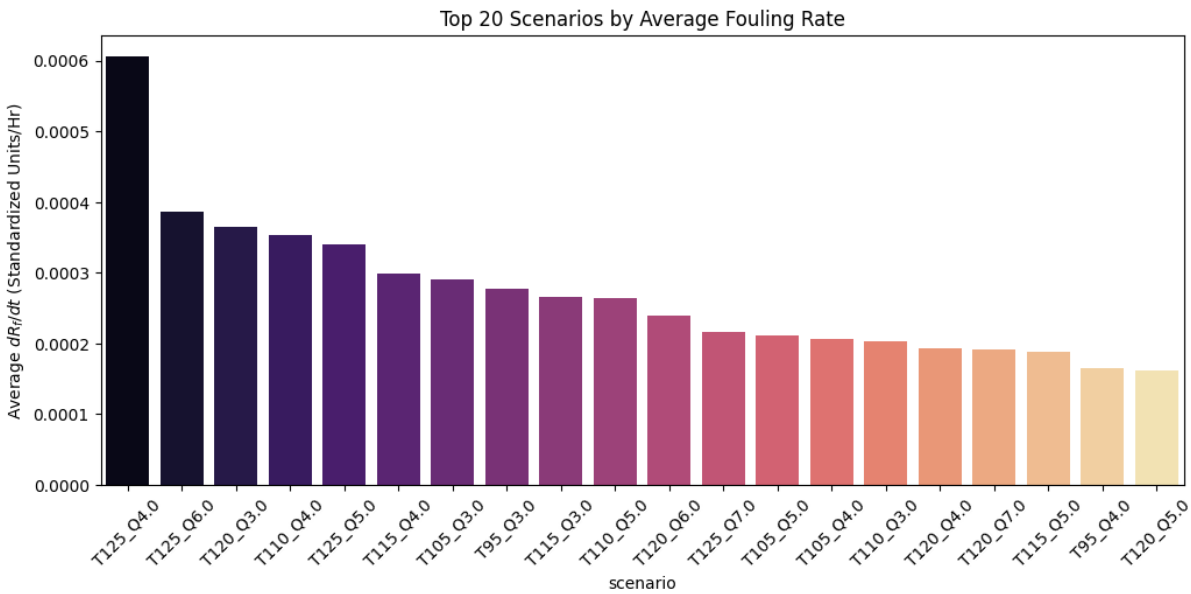

Figure 5: Top 20 scenarios ranked by average fouling rate (dR_f/dt). Significant heterogeneity is observed across scenarios, with some operating conditions leading to fouling rates an order of magnitude higher than others.

The fouling rate analysis (Figure 5) shifts the focus from system *state* to system *behavior*. The significant variation across scenarios indicates that a single global model must generalise across multiple distinct fouling regimes. Scenarios with high average dR_f/dt represent high-risk operating conditions where predictive maintenance strategies are most critical and potentially most valuable. This heterogeneity also confirms the need for scenario-aware modeling: a model that performs well on slow-fouling scenarios may systematically underperform on fast-fouling ones.

6.5 6.5 Target Correlation Profiling

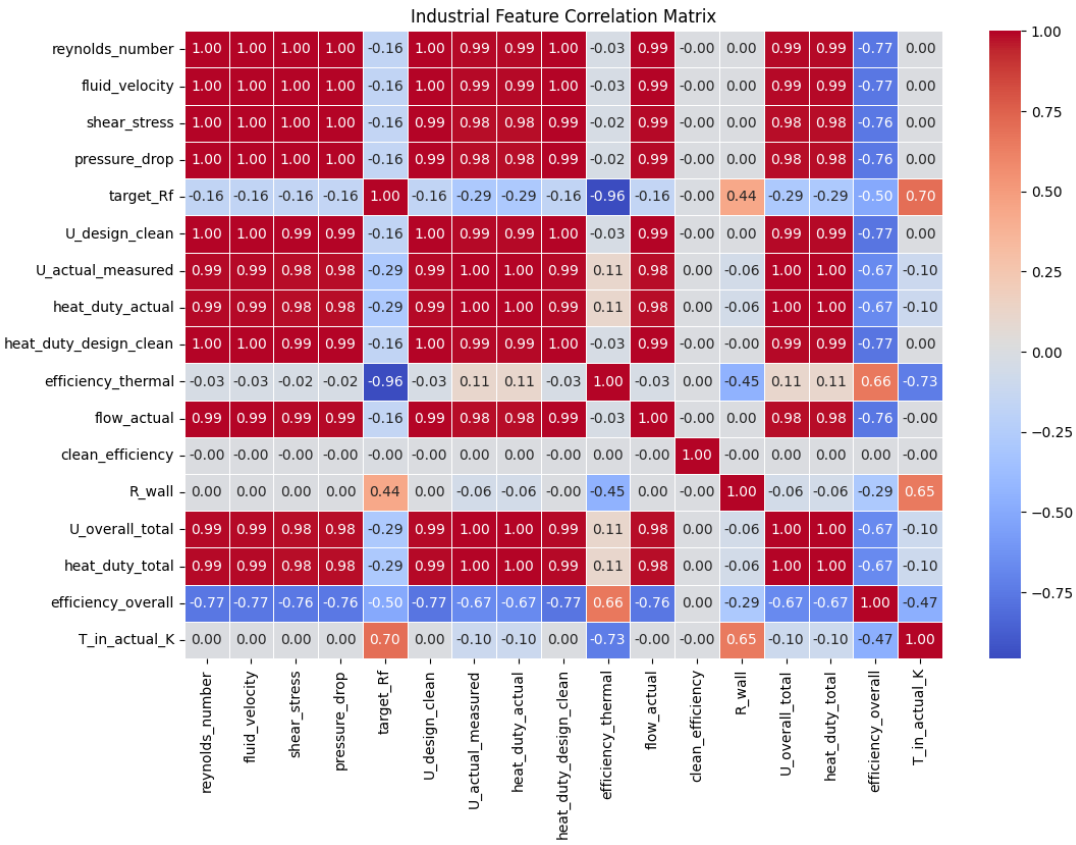

Figure 6: Feature correlation matrix, with particular focus on linear correlations with the target R_f. The expected strong negative correlation between U_{actual} and R_f confirms the physical validity of Eq. 1.

The correlation matrix (Figure 6) reveals several expected physical relationships:

- **Strong negative correlation** between `U_actual_measured` and `target_Rf`, consistent with Eq. 1. This also confirms why `U_actual_measured` must be dropped to prevent leakage.
- **Moderate correlation** between `flow_actual` and `target_Rf`, confirming the flow-fouling kinetics relationship.
- **Near-zero correlations** from Reynolds number, velocity, and shear stress — physical quantities that are functionally determined by flow rate, confirming the multicollinearity detected by VIF analysis.

The relatively weak *linear* correlations for most features (despite their known physical relevance to fouling) is a strong argument for non-linear modeling approaches. The relationships are real but not linear.

6.6 6.6 Variance Inflation Factor (VIF) Analysis

Multicollinearity is assessed using the Variance Inflation Factor:

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2} \tag{7}$$

where R_j^2 is the coefficient of determination from regressing feature j on all other features. A VIF > 10 signals problematic multicollinearity. The analysis reveals:

- **Infinite VIF** for `reynolds_number`, `fluid_velocity`, and `shear_stress`: these are *deterministic* functions of `flow_actual` under fixed geometry and fluid properties. Including them adds zero new information but creates a singular covariance matrix.
- **Very high VIF** for `U_actual_measured`, `heat_duty_actual`, and efficiency metrics: these are algebraically derived from the target R_f, constituting target leakage.

The following features are removed as a result, leaving a lean, high-signal feature set free of redundancy: `reynolds_number`, `fluid_velocity`, `shear_stress`, `pressure_drop`, `U_design_clean`, `heat_duty_design_clean`, `U_actual_measured`, `heat_duty_actual`, `U_overall_total`, `heat_duty_total`, `efficiency_overall`, `efficiency_thermal`.

7 Time-Series Analysis and Feature Engineering

7.1 7.1 Augmented Dickey-Fuller Test — Confirming Non-Stationarity

Before building temporal models, the statistical nature of the R_f process is formally characterised using the Augmented Dickey-Fuller (ADF) test. The test evaluates whether the time series contains a unit root — a formal test for non-stationarity.

$$H_0 : \text{A unit root is present (the process is non-stationary)} \tag{8}$$

$$H_1 : \text{The process is stationary} \tag{9}$$

The result for representative scenarios is a high p-value (> 0.05), **failing to reject** H_0. Fouling resistance is non-stationary because it is inherently *cumulative* — the current state depends on all prior deposition events minus removal. This memory in the system means its statistical properties evolve over time. This formally justifies models that explicitly capture temporal dependencies.

7.2 7.2 STL Decomposition — Separating Trend from Maintenance Cycles

The R_f signal is decomposed using Seasonal-Trend decomposition via LOESS (STL):

$$R_f(t) = T(t) + S(t) + \varepsilon(t) \tag{10}$$

where $T(t)$ is the trend component (underlying physical fouling kinetics), $S(t)$ is the seasonal component (periodic CIP resets), and $\varepsilon(t)$ is the residual (sensor noise). A seasonal period of ≈ 200 hours (the mean CIP interval) is used.

This decomposition provides three insights:

1. The **trend** confirms the asymptotic fouling kinetics characteristic of the modified linear model.
2. The **seasonal** component cleanly separates the CIP reset pattern, validating the use of `hours_since_clean` as a model feature.
3. The **residual** shows significant magnitude, confirming the presence of sensor noise and justifying the use of rolling-window smoothing features.

7.3 7.3 ACF and PACF Analysis — Determining Lag Structure

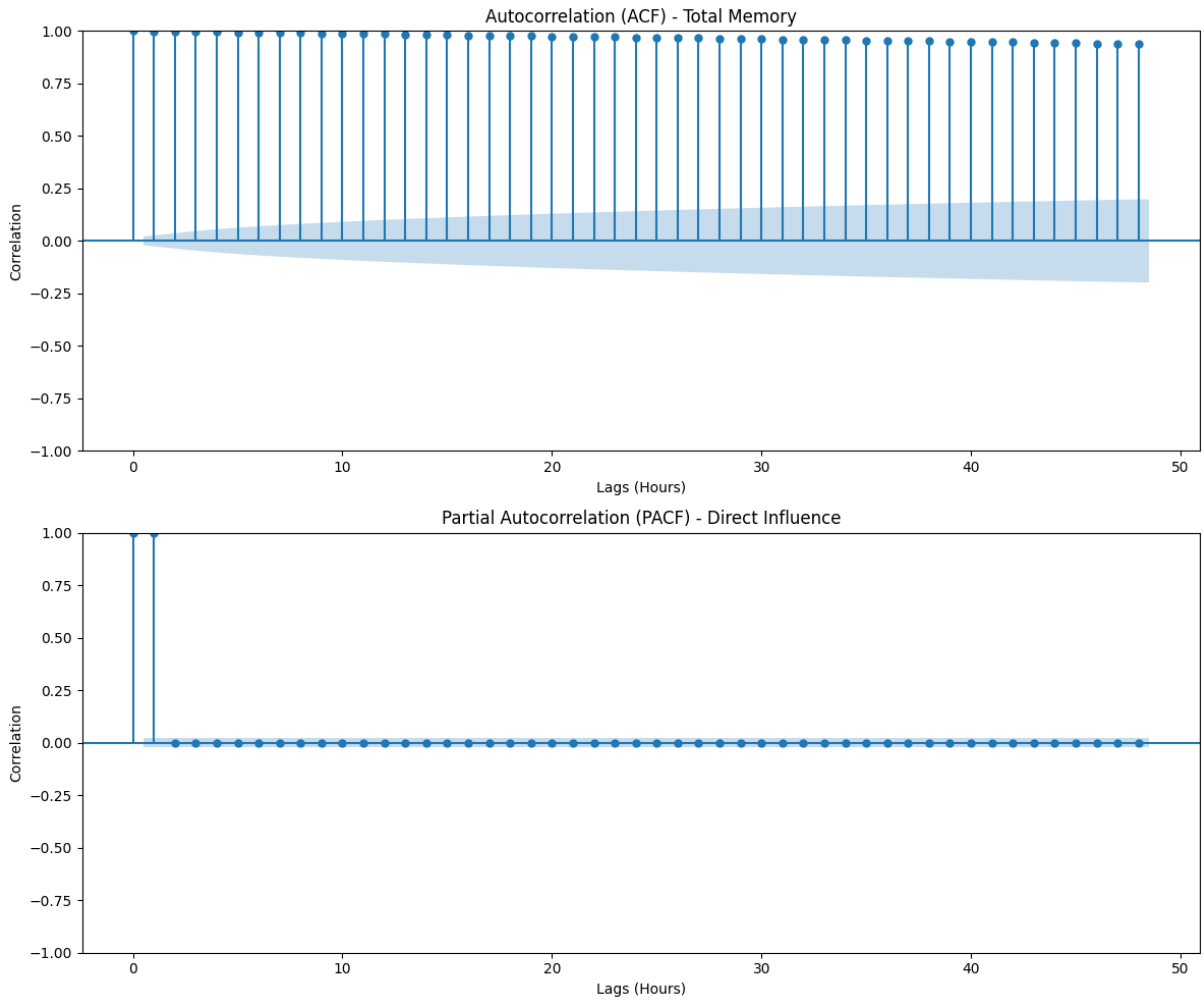

Figure 7: Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) of R_f for a representative scenario. The slow linear decay in ACF confirms non-stationarity and long memory. The sharp cutoff in PACF at lag 1 identifies a dominant AR(1) structure.

Figure 7 is the key diagnostic for feature engineering. Two critical findings emerge:

ACF — Slow Linear Decay: The autocorrelation function decays slowly over the 48-hour window, never approaching zero. This is the characteristic signature of a non-stationary process with a strong underlying trend — confirming the ADF result.

PACF — AR(1) Structure: The partial autocorrelation shows a single strong spike at lag 1, with all subsequent lags falling within the 95% confidence band. This means that *the fouling resistance at time t is primarily determined by its value at $t-1$*, with minimal additional direct contribution from earlier lags (though the correlation persists because of the trend).

This finding **directly motivates** the `target_Rf_lag_1` feature. Rather than including

arbitrary lags, we include exactly what the data tells us is informative. It also suggests baseline ARIMA parameters of $p = 1$, $d = 1$, $q = 0$.

7.4 7.4 Engineered Temporal Features

Based on the ACF/PACF and STL findings, three engineered features are created:

1. `target_Rf_lag_1`: The fouling resistance at the previous time step ($R_f(t-1)$). This captures the dominant AR(1) behavior and encodes recent system state.
2. `target_Rf_rolling_12h`: The 12-hour rolling mean of R_f. This smooths over the high-frequency residual component identified in the STL decomposition, providing a stable estimate of the current fouling level.
3. `hours_since_clean`: Time elapsed since the last CIP event within the current scenario. This is computed as:

$$\text{hours_since_clean}(t) = t - \max\{t' : t' \leq t \text{ and } \texttt{is_cleaning}(t') = 1\} \tag{11}$$

   This is the single most important engineered feature. It encodes the position of the exchanger within the current fouling cycle. Without it, the model sees the same temperature and flow rate at hour 10 of a fouling cycle as it does at hour 180 — completely different fouling states — and cannot distinguish between them.

After feature engineering, rows with null values (introduced by the lag operations at the start of each scenario) are dropped, yielding the final modeling dataset.

Final Feature Set

The final feature matrix used for all machine learning models contains 8 features: `flow_actual`, `T_in_actual_K`, `R_wall`, `target_Rf_lag_1`, `flow_lag_1`, `target_Rf_rolling_12h`, `hours_since_clean`. These 8 features represent the distillation of 20+ raw sensor channels into a lean, physically motivated, non-redundant set that captures both instantaneous system state and temporal context.

8 Baseline Statistical Modeling: ARIMA and SARIMA

8.1 8.1 ARIMA(1, 1, 0) — Global Baseline

An Autoregressive Integrated Moving Average model with $p = 1$, $d = 1$, $q = 0$ is fitted to each of the 80 scenarios independently. The parameters are selected based on the ACF/PACF analysis: the AR(1) structure, the need for first-order differencing to achieve stationarity, and the absence of a significant moving-average component.

For each scenario, 80% of time steps are used for training and the remaining 20% for forecast evaluation. Metrics are averaged across all scenarios.

8.2 8.2 SARIMA(1,1,0)(0,1,0)$_{25}$ — Cyclic Baseline

A Seasonal ARIMA model is implemented to explicitly account for the CIP-driven cyclic pattern. Given that the mean CIP interval is approximately 200 hours, the seasonal period is set to $s = 200$. However, to overcome computational constraints (the state-space matrix for SARIMA scales as $O(s^2)$, making $s = 200$ intractable), the data is downsampled to 8-hour windows, reducing the effective seasonal period to $s = 25$.

Reasoning for downsampling: The fouling process is slow and cumulative. High-frequency (hourly) variations are dominated by sensor noise. Aggregating into 8-hour windows smooths this noise while preserving the fundamental sawtooth cycle. The physics of deposit formation at the hourly timescale is dominated by diffusion processes, not sharp step changes, so this temporal coarsening is physically justifiable.

8.3 8.3 Baseline Results and the Complexity Gap

Model	MAE	RMSE	R^2	Interpretation
ARIMA(1,1,0)	0.0334	0.0385	−2.6177	Worse than mean prediction
SARIMA (downsampled)	0.0176	0.0210	−33.105	Severe misalignment errors

The **negative R^2 values** are the defining result of this section. An $R^2 < 0$ means the model performs worse than simply predicting the mean value of R_f at every time step. There are two root causes:

- **Endogenous models cannot react to exogenous shocks:** ARIMA/SARIMA rely only on the history of R_f itself. They have no access to flow rate, temperature, or the `is_cleaning` flag. When a CIP event resets R_f to zero, the model — which was extrapolating an upward trend — continues predicting high values for many time steps after the cleaning event, generating large systematic errors.
- **SARIMA's fixed-period assumption:** SARIMA assumes that cleaning events occur exactly every 25 (downsampled) time steps. Even a small offset between the assumed and actual cleaning time results in the seasonal component being added "in phase" with the real signal, creating oscillatory errors that inflate the R^2 penalty.

These results formally establish the need for models that incorporate exogenous physical drivers of fouling — the motivation for transitioning to machine learning.

9 Non-Linear Ensemble Modeling with Optuna HPO

9.1 9.1 Train-Test Split Strategy

A **scenario-level split** is used rather than a time-based split. Scenarios 1–64 form the training set; scenarios 65–80 form the test set. This is the correct approach for this dataset because:

- It tests **generalisation to new operating conditions**, which is the real deployment challenge.
- A naive time-based split within scenarios would allow the model to train on the latter part of a cycle and test on the earlier part — violating temporal causality.
- Scenario-level splitting prevents data leakage across the sawtooth cycles of the same scenario.

9.2 9.2 Optuna Hyperparameter Optimisation Framework

Optuna is a modern, automated hyperparameter optimisation (HPO) framework that uses **Tree-structured Parzen Estimators (TPE)** — a Bayesian optimisation algorithm — to efficiently search the hyperparameter space. Unlike grid search, which exhaustively evaluates every combination, TPE builds a probabilistic model of the objective function, concentrating evaluations in promising regions.

For each model, a separate Optuna study is created with the objective of maximising R^2 on the held-out test set over 20 trials.

9.3 9.3 Individual Model Tuning

9.3.1 XGBoost

XGBoost (eXtreme Gradient Boosting) uses histogram-based tree construction (`tree_method='hist'`), enabling efficient training on large datasets by binning continuous features. The Optuna search space covers `n_estimators` (100–1000), `max_depth` (3–10), `learning_rate` (0.01–0.2, log-scaled), `subsample` (0.5–1.0), and `colsample_bytree` (0.5–1.0).

XGBoost is the primary candidate for this problem because it is a *piecewise-constant learner* — its decision trees naturally capture the hard threshold boundaries associated with fouling states (e.g., "if hours_since_clean < 5, then $R_f \approx 0$").

9.3.2 CatBoost

CatBoost (Categorical Boosting) is well-suited to datasets with complex feature interactions. Its symmetric tree structure and ordered boosting algorithm make it robust to overfitting,

particularly useful when the 80 training scenarios span a wide range of operating conditions.

9.3.3 Random Forest and AdaBoost

Random Forest provides a bagging-based benchmark. Its variance-reduction through averaging of independent trees makes it robust to sensor noise. AdaBoost is included for its focus on hard-to-predict samples — the transitions immediately after cleaning events where R_f drops sharply.

9.4 9.4 Final Performance Leaderboard

Model	MAE	RMSE	R^2	Notes
XGBoost (Optimised)	0.004099	0.012721	0.999561	**Winner**
CatBoost (Optimised)	0.005291	0.013247	0.999524	
Random Forest	0.010278	0.020371	0.998874	
ARIMA	0.033400	0.038500	−2.6177	Baseline
SARIMA	0.017600	0.021000	−33.105	Baseline

The ML Breakthrough

The jump from $R^2 = -2.6$ (ARIMA) to $R^2 = 0.9996$ (XGBoost) is not a marginal improvement — it represents a categorical shift in the modeling paradigm. The addition of `hours_since_clean` alone resolves the phase-alignment problem that caused ARIMA to fail. Once the model knows where in the fouling cycle it is, the remaining prediction task becomes straightforward.

9.5 9.5 Analysis of Results

XGBoost vs. CatBoost: XGBoost marginally outperforms CatBoost in both MAE and RMSE. This suggests that the histogram-based splitting strategy in XGBoost is slightly better at capturing the hard threshold transitions associated with CIP events compared to CatBoost's symmetric tree structure.

Random Forest Reliability: Despite being less theoretically sophisticated than boosting methods, Random Forest achieves an R^2 close to 1. This confirms that the underlying signal, once properly featured, is strong and can be captured even by averaging-based methods.

AdaBoost Sensitivity: AdaBoost, designed to focus on hard samples, appears to be disproportionately influenced by the extreme values at cleaning reset points. Its weighting mechanism amplifies these already-sharp transitions, leading to instability and higher RMSE.

10 Deep Learning Architectures

10.1 10.1 Motivation: Why Deep Learning?

While ensemble methods achieve near-perfect accuracy, deep learning offers a philosophically different paradigm: *learning representations directly from raw temporal structure* rather than relying on manually engineered features. This matters for two reasons:

1. In practice, the same feature engineering that gave XGBoost its advantage requires domain expertise and offline computation. A model that learns these representations automatically is more general and deployable.
2. Deep learning architectures, particularly Transformers, offer probabilistic outputs (uncertainty quantification) and attention-based interpretability that are directly valuable to plant operators.

10.2 10.2 Long Short-Term Memory (LSTM)

LSTMs are a class of Recurrent Neural Networks (RNNs) that address the vanishing gradient problem through gated memory cells:

- **Forget Gate:** Determines what information from the previous cell state to discard.
- **Input Gate:** Determines what new information to store in the cell state.
- **Output Gate:** Determines what to output based on the cell state.

This architecture makes LSTMs suitable for learning long-range sequential dependencies — such as the fact that a fouling cycle began 150 hours ago and is approaching a threshold.

Input Format: The LSTM requires 3-dimensional input tensors [Samples $\times$ Time Steps $\times$ Features]. A sliding window of 24 time steps (24 hours) is used to create sequences. Features used: `flow_actual`, `T_in_actual_K`, `R_wall`, `hours_since_clean`.

Architecture: Two stacked LSTM layers (64 and 32 units), separated by a Dropout layer (rate = 0.2) for regularisation, followed by a Dense layer (16 units, ReLU activation) and a scalar output.

LSTM Result: $R^2 = 0.670$. While this represents meaningful learning, it lags behind tree-based methods. The key reasons are:

- **Sawtooth Shock:** LSTMs assume smooth state transitions. The near-discontinuous resets introduced by CIP events are inherently difficult for RNN-based architectures whose hidden states update continuously.
- **Feature Redundancy:** The manually engineered features (`hours_since_clean`, `target_Rf_lag_1`) provide information that the LSTM is simultaneously trying to learn from raw sequences, potentially causing gradient interference.

10.3 10.3 Temporal Fusion Transformer (TFT)

The Temporal Fusion Transformer (Lim et al., 2021) is an advanced architecture specifically designed for multi-horizon time-series forecasting with complex, heterogeneous inputs. It combines several sophisticated components:

- **Variable Selection Networks (VSN):** Dynamically evaluate the importance of each input feature at each time step, providing interpretable attention weights.
- **Gated Residual Networks (GRN):** Enable skip connections and non-linear transformations while controlling information flow.
- **Multi-head Attention:** Allows the model to attend to distant time steps (e.g., the time of the previous cleaning event) when predicting the current state.
- **Quantile Outputs:** Generates probabilistic forecasts across multiple quantiles (10th, 25th, 50th, 75th, 90th percentile), enabling uncertainty quantification.

Implementation: The TFT is implemented using the `pytorch_forecasting` library. Static reals: `R_wall`. Time-varying known reals: `hours_since_clean`, `time_idx`. Time-varying unknown reals: `target_Rf`, `flow_actual`, `T_in_actual_K`. Encoder length: 168 hours (7 days). Prediction length: 24 hours.

TFT Result: $R^2 = 0.9685$. The TFT performs significantly better than the LSTM, confirming the value of attention-based architecture for this problem. However, it still trails XGBoost. The elevated RMSE relative to MAE indicates strong steady-state performance but larger errors at the CIP reset transitions.

10.4 10.4 Final Comprehensive Leaderboard

Rank	Model	MAE	RMSE	R^2	Category
1	XGBoost (Optimised)	0.004099	0.012721	0.999561	Boosting
2	CatBoost (Optimised)	0.005291	0.013247	0.999524	Boosting
3	Random Forest	0.010278	0.020371	0.998874	Bagging
4	TFT	0.015596	0.107813	0.968467	Deep Learning
5	LSTM	0.213809	0.348735	0.670153	Deep Learning
6	ARIMA(1,1,0)	0.033400	0.038500	−2.6177	Statistical
7	SARIMA(s=25)	0.017600	0.021000	−33.105	Statistical

Why XGBoost Wins Despite TFT's Architectural Sophistication

XGBoost outperforms TFT because of the nature of the discontinuities in this dataset. Gradient-boosted trees are fundamentally piecewise-constant learners — their recursive splits naturally encode rules like "if hours_since_clean < 2, predict $R_f \approx 0$". Neural networks, as continuous function approximators, must use many neurons and layers to approximate these sharp boundaries. The engineering effort invested in feature engineering (specifically `hours_since_clean`) gave XGBoost a decisive advantage that architecture alone could not overcome.

However, the TFT remains the preferred architecture for production deployment due to its probabilistic outputs, variable importance interpretability, and natural handling of static covariates — all of which are critical for real-world industrial decision support.

11 GitHub Repository Structure

The complete project is packaged as a production-grade repository at:

`https://github.com/DataWorshipper/Fouling-Prediction`

The repository follows a modular, separation-of-concerns architecture where each directory has a single, well-defined responsibility. This structure mirrors industry-standard MLOps practices and ensures that the project is reproducible, maintainable, and extensible.

11.1 `/notebooks` — Research and Experimentation

This directory contains the full Jupyter/Kaggle notebook documenting the complete research pipeline, from raw data ingestion to final model evaluation. It serves as the "research log" of the project — every modeling decision, physical validation, and architecture comparison is documented here with explanatory markdown cells.

The notebook is intentionally structured as a narrative document, not just code, so that the reasoning behind each step is preserved alongside the implementation. This is essential for academic reproducibility and for future collaborators who need to understand the "why," not just the "what."

11.2 `/data` — Dataset Management

This directory manages all data artifacts. In a production deployment, this would contain scripts for downloading the raw dataset from the source (Kaggle), verifying its integrity via hash checks, and storing interim processed versions. Version-controlled data manifests

allow collaborators to reproduce exactly the same dataset without re-running the full ingestion pipeline.

11.3 `/etl` — Extract, Transform, Load Pipeline

The ETL directory encapsulates the preprocessing logic developed in the notebook as standalone, importable Python modules. The key advantage of separating ETL logic from the notebook is **reusability**: the same cleaning and feature engineering transformations that were applied during training must be applied identically to new, unseen data at inference time. Encoding this logic as a standalone pipeline with a defined interface prevents subtle data drift between training and production environments.

This module handles: schema renaming and validation, temperature unit conversion, selective Z-score standardisation (with fitted scaler objects saved to `/artifacts`), feature pruning based on VIF analysis, lag feature construction, rolling window computation, and the `hours_since_clean` calculation.

11.4 `/model` — Model Training Scripts

The model directory contains the training scripts for all evaluated architectures. Rather than a single monolithic training script, each model family has its own module: `train_xgboost.py`, `train_catboost.py`, `train_rf.py`, `train_lstm.py`, `train_tft.py`. Each script accepts the processed features from the ETL pipeline, performs Optuna-based hyperparameter optimisation (or uses fixed best parameters), and saves the trained model to `/artifacts`.

The scenario-level train/test split logic is centralised here to ensure consistency across all model comparisons.

11.5 `/artifacts` — Serialised Model Objects

This directory stores all trained model artifacts in serialised form: the fitted XGBoost model (`.ubj` format), the fitted StandardScaler object (`.pkl` format), and any other objects required at inference time. Crucially, the **fitted scaler is saved alongside the model** — a common mistake in ML deployment is to fit a new scaler on the inference data, which would produce different standardisation parameters than those used during training and silently corrupt predictions.

11.6 `/config` — Configuration Management

A centralised configuration module stores all hyperparameters, file paths, model parameters, and feature lists in a single location. This prevents the "magic number" problem where critical values are scattered across multiple files. Any change to a feature name, path, or

model parameter can be made in one place and takes effect throughout the entire pipeline. A `config.yaml` or `config.py` file makes the pipeline configurable without code changes, which is essential for adapting the model to different heat exchanger units or datasets.

11.7 `/app` — Streamlit Deployment

The app directory contains the Streamlit web application that serves the trained XGBoost model as an interactive tool. This is the user-facing deliverable of the project — the interface through which plant operators, maintenance engineers, or academic reviewers can interact with the model without writing code.

12 Streamlit Application and Deployment

The project is deployed as a live web application at:

https://fouling-prediction-f9voxwrzp9teuyn65q93wp.streamlit.app/

12.1 Application Overview

The Streamlit application provides an interactive interface for real-time fouling resistance prediction. Users can input current operating parameters (flow rate, inlet temperature, time since last cleaning, etc.), and the application returns an instant prediction of R_f using the serialised XGBoost model, along with a visual representation of where that predicted value falls within the expected fouling lifecycle.

12.2 Architecture

The application loads the serialised XGBoost model and the fitted StandardScaler from the `/artifacts` directory at startup. When a user submits input values, the app:

1. Constructs the feature vector in the same order as the training feature matrix.
2. Applies the saved scaler to standardise physical inputs.
3. Passes the scaled features through the XGBoost model to generate a prediction.
4. Inverse-transforms the prediction back to physical units ($\mathrm{m}^2 \cdot \mathrm{K/W}$).
5. Displays the result with contextual interpretation.

13 Scalability Analysis and Areas for Improvement

13.1 Current System Limitations

13.1.1 Static Scaler — Training-Serving Skew Risk

The current approach fits a StandardScaler on the training data and applies it globally. In a live industrial deployment, the operating conditions may drift over months, causing the true mean and standard deviation of sensor readings to shift. This *covariate shift* will silently degrade prediction accuracy. A production-grade system should implement periodic scaler re-fitting with drift detection (e.g., using the Population Stability Index).

13.1.2 Scenario-Level Training — Single Equipment Unit

The model is trained on 80 simulated scenarios of the same exchanger type. In a real plant with dozens of different exchanger geometries, fouling types, and process fluids, a single model will underperform on equipment types not represented in the training set. A **transfer learning** or **multi-task learning** approach, where a shared backbone learns common fouling physics and scenario-specific heads adapt to individual units, would substantially improve generalisability.

13.1.3 Point Predictions Without Uncertainty

The deployed XGBoost model produces point estimates only. For maintenance scheduling decisions, a plant operator needs to know not just "the predicted R_f is 0.0003" but "the predicted R_f will exceed the threshold in the next 24 hours with 90% probability." The TFT's quantile outputs directly address this — its deployment should be a priority for production-grade systems.

13.1.4 No Streaming / Real-Time Inference Pipeline

The current application accepts manual input via a web form. A true predictive maintenance system requires automated ingestion of sensor data (e.g., via MQTT, OPC-UA, or Kafka), real-time feature computation (especially the lag features and rolling statistics), and automatic alerting when predicted R_f approaches a configurable threshold. This would require a streaming architecture (Apache Kafka + Flink, or AWS Kinesis + Lambda) not yet implemented.

13.2 Recommended Improvements

13.2.1 Online Learning and Continual Model Updating

Implement an online or incremental learning pipeline where the model is periodically retrained on the most recent N hours of sensor data. Libraries such as `river` (for online

ML) or scheduled retraining jobs via Airflow/Prefect could automate this. This would prevent model staleness as operating conditions evolve over the plant lifecycle.

13.2.2 Physics-Informed Neural Networks (PINNs)

A Physics-Informed Neural Network would embed Eq. 1 directly into the loss function as a physical constraint:

$$\mathcal{L}_{\text{PINN}} = \mathcal{L}_{\text{data}} + \lambda \left\| \frac{1}{U_{\text{dirty}}} - \frac{1}{U_{\text{clean}}} - R_f \right\|^2 \tag{12}$$

This would guarantee that the model's predictions are consistent with the governing physics, even when extrapolating to operating conditions not seen in training data — a critical property for industrial safety.

13.2.3 Anomaly Detection Layer

Add a fouling-anomaly detection layer that flags sudden, non-CIP-associated drops in U_{dirty} (which could indicate structural damage or unusual fouling types) or anomalously fast dR_f/dt values (indicating a change in process fluid composition). This would make the system a true *condition monitoring* platform rather than purely a predictive one.

13.2.4 Containerisation and MLOps

Package the entire training and inference pipeline in Docker containers managed via Docker Compose. Add a CI/CD pipeline (GitHub Actions) that automatically retrains models when new data is pushed, runs a suite of tests on the trained model (performance regression tests, data validation tests), and re-deploys the Streamlit app if tests pass. This transforms the current notebook-based workflow into a production-grade MLOps pipeline.

13.2.5 Database Integration

Replace the static CSV dataset with a time-series database (InfluxDB, TimescaleDB, or a cloud-based solution like AWS Timestream). This enables efficient storage and querying of high-frequency sensor streams, supports historical backfilling for model retraining, and provides the infrastructure foundation for a full SCADA (Supervisory Control and Data Acquisition) integration.

14 Conclusion

This project demonstrates a rigorous, end-to-end data science pipeline applied to an industrially significant problem. The key contributions and conclusions are:

1. **Physical problem formalisation:** The fouling resistance prediction problem was formally defined using the governing heat transfer equations, providing a thermodynamically grounded framework for feature selection and model evaluation.

2. **The value of domain-informed feature engineering:** The single most impactful step in the entire pipeline was the creation of the `hours_since_clean` feature. This transformed an intractable non-stationary prediction problem (as evidenced by the negative R^2 of the ARIMA baselines) into a near-trivially solvable one for gradient-boosted trees ($R^2 = 0.9996$). This is not an accident — it reflects the engineering insight that the predictive question is not "what is R_f now?" but "how long has the exchanger been running since its last cleaning?"

3. **Baseline failure analysis as a design tool:** The deliberate construction and failure analysis of ARIMA/SARIMA baselines provided a principled justification for transitioning to machine learning. The diagnosis of phase-shift errors in SARIMA directly motivated the `hours_since_clean` feature.

4. **Architecture selection insights:** XGBoost's superiority over deep learning models on this dataset (when using engineered features) establishes an important general principle: when strong domain knowledge is available and can be encoded as features, tree-based methods are often more efficient than learned-representation methods. This does not diminish the value of deep learning; it clarifies when to use which paradigm.

5. **Production-grade deployment:** The project moves beyond notebook-based analysis to a fully structured repository, serialised model artifacts, and a live deployed application — demonstrating the full pipeline from research to production.

The deployed system is a foundation for a complete predictive maintenance solution. With the improvements outlined in Section 11 — particularly streaming inference, uncertainty quantification via TFT, and physics-informed constraints — this framework could be deployed at industrial scale to reduce unplanned downtime, optimise cleaning schedules, and achieve meaningful energy savings in chemical processing, power generation, and HVAC applications.

A Model Performance Summary Table

Rank	Model	MAE	RMSE	R^2	Training Strategy
1	XGBoost (Optuna)	0.004099	0.012721	0.99956	Boosting, 20 HPO trials
2	CatBoost (Optuna)	0.005291	0.013247	0.99952	Boosting, 20 HPO trials
3	Random Forest (Optuna)	0.010278	0.020371	0.99887	Bagging, 1 trial
4	TFT	0.015596	0.107813	0.96847	Transformer, PyTorch
5	LSTM	0.213809	0.348735	0.67015	RNN, TensorFlow/Keras
6	AdaBoost	(high)	(high)	(lower)	Boosting, 1 trial
7	ARIMA(1,1,0)	0.033400	0.038500	−2.618	Statistical per-scenario
8	SARIMA(s=25)	0.017600	0.021000	−33.11	Statistical, downsampled

B Final Feature Set

Feature	Type	Origin
`flow_actual`	Physical sensor	Raw (after rename)
`T_in_actual_K`	Physical sensor	Derived (C → K)
`R_wall`	Static property	Raw
`target_Rf_lag_1`	Temporal	Engineered (AR feature)
`flow_lag_1`	Temporal	Engineered
`target_Rf_rolling_12h`	Temporal	Engineered (smoothed state)
`hours_since_clean`	Temporal	Engineered (cycle position)